5-3 · Iceberg · MinIO 연결 · 실습 1.5H

Spark ↔ MinIO(s3a) 연결

MinIO 를 붙이고, Parquet 왕복으로 확인

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① 5-2 compose 끝에 MinIO 를 한 번 붙여 컨테이너 4개를 Up 으로

② warehouse 버킷을 만들고 lake-pyspark.sh 로 셸을 띄운다

③ s3a 로 Parquet 을 쓰고 읽어 count 100 · 파티션 4 를 본다

④ 버킷 · 열쇠 · 주소 · path style 오류 네 가지를 구분한다

여기서 막히면 Iceberg 문제가 아니라 연결 문제 — 다음 1H 전에 왕복까지 끝냅니다

실습 흐름

다섯 단계 — 5-2 compose 위에서

4주차 MinIO 는 쓰지 않습니다 — 9000 · 9001 이 겹치니 stop 만 (지우지 않음)

1. compose	2. 버킷	3. PySpark	4. 왕복	5. 일부러 틀리기
cat >> 로 minio 덧붙이기	mc mb warehouse	lake-pyspark.sh 첫 실행 1~2분	count 100 파티션 4	재시도 1번으로 오류 네 가지

1·2단계

MinIO 붙이기 — cat >> 는 딱 한 번

1 **cat >> docker-compose.yml << 'EOF'**
minio 덩이(2칸) + volumes 덩이(0칸)를 파일 끝에 덧붙인다

2 **docker compose config --services**
spark-master · spark-worker · minio 세 줄

3 **4주차 stop → up -d → ps**
컨테이너 4개 Up — spark-master · spark-minio · worker-1 · 2

4 **mc mb -p m/warehouse**
Bucket created successfully — 네트워크 week5_default

두 번 붙이면

volumes

already defined

3단계

lake-pyspark.sh — 열두 줄을 파일 하나로

1 **cat > lake-pyspark.sh << 'EOF'**
exec spark-master pyspark + --master · --packages · --conf

2 **--conf spark.jars.ivy=/tmp/.ivy2**
받은 jar 를 둘 곳 — 이 줄이 없으면 바로 끝난다

3 **s3a 4줄 + Iceberg 4줄**
endpoint http://minio:9000 · admin · warehouse=s3a://warehouse/lake

띄우기

bash

lake-pyspark.sh

endpoint 는 localhost 가 아니라 minio(서비스 이름) — 셸이 컨테이너 안에서 돈다

3단계 — 왜 spark.jars.ivy 인가

jar 를 둘 곳 · 받는 데 걸리는 시간

빼면 — 바로 실패

apache/spark 이미지의 spark 사용자는 홈 폴더가 없다

--packages	/home/spark 없음	FileNotFound
~/.ivy2 에 받으려 함	둘 곳이 없다	.../resolved-...xml

VS

넣으면 — /tmp/.ivy2

캐시는 컨테이너 안 — stop 뒤 남고 down 하면 다시 받음

첫 실행 1~2분	aws-java-sdk	두 번째 10~20초
jar 4개 316MB	혼자 74.6초	4 already retrieved

4단계

s3a 왕복 — count 100 이면 연결 완료

쓰기 → 읽기

Iceberg 전에 「그냥 파일 읽고 쓰기」부터

spark.range(1, 101)	write.parquet	read → count
100행	smoke-test/	100

VS

파일 수 = 파티션 수

셸에서는 4 — 워커 둘의 코어 4개만큼

getNumPartitions	smoke-test/	coalesce(1)
4	part 4개	smoke-one/ 1개

5단계 — 일부러 실패시키기

셸 안에서 다시 시도를 1번으로 줄이고 틀린다

1. **setLogLevel("ERROR")**
   빼면 S3A 재시도 경고가 자바 스택 수십 줄로 결과를 덮는다

2. **try_read("s3a://no-such-bucket/x/")**
   없는 버킷은 1초 만에 404 — 재시도 없이 바로

3. **attempts.maximum · retry.limit = 1**
   + impl.disable.cache=true — 하나씩 틀렸다 같은 줄에서 되돌림

그냥 두면 주소는

약 15분

열쇠 약 2.5분

끝나면 성공: 100 → 세 설정 unset → setLogLevel("WARN") 로 원래대로

5단계 — 일부러 실패시키기

틀린 것마다 찍히는 문구가 다르다 (실측)

틀린 것	찍히는 문구	시간	처음 볼 곳
버킷 이름	UnknownStoreException NoSuchBucket (404)	1초	콘솔의 버킷 목록
열쇠(secret)	AccessDeniedException 403 Forbidden	0초	MINIO_ROOT 값
주소(endpoint)	Unable to execute HTTP request: mini0	3초	철자 · 서비스 이름
path style	Unable to execute HTTP request: warehouse.minio	2초	path.style.access

셸이 몇 분 말없이 멈추면 고장이 아니라 재시도 — Ctrl+C 뒤 endpoint · 열쇠 · path style 부터

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. docker compose ps 에 컨테이너 4개가 Up 이다
2. localhost:9001 에 admin / password123 으로 들어가진다
3. bash lake-pyspark.sh 로 >>> 가 뜬다 (두 번째부터 10~20초)
4. count 100 · 파티션 4 · smoke-test part 4개 · smoke-one 1개
5. 네 가지 오류를 문구로 구분하고, 마지막은 성공: 100

5-3 · 실습 1.5H 마무리

다음은 Iceberg 카탈로그 구성과 테이블 생성 (1H)

띄워 둔 셸에서 SQL 로 Iceberg 테이블을 만들고, MinIO 에 어떤 파일이 생기는지 확인합니다

- ✔ **CREATE NAMESPACE lake.db**
- ✔ **CREATE TABLE … USING iceberg · INSERT**
- ✔ **콘솔 lake/db/orders 의 metadata · data**
- ✔ **snapshots 메타테이블과 타임 트래블**

가져갈 것

띄워 둔 PySpark 셸(>>>) 그대로